CORPO DE BOMBEIROS MILITAR DE MINAS GERAIS

API CAD
Manual de Integração

Guia direto para desenvolvedores que vão acoplar a leitura de chamadas e recursos ativos do CAD ao próprio sistema.

`https://apicadbm.bombeiros.mg.gov.br`

TRÊS REGRAS ANTES DE QUALQUER LINHA DE CÓDIGO

1 **O token chega por e-mail e não aparece de novo.**
Não há cadastro nem tela para gerar token. Guarde em cofre ou variável de ambiente. Nunca na URL, em log, em print ou em repositório.

2 **Tabelas auxiliares só traduzem a API principal.**
Não são uma segunda API operacional. Carregue uma vez, guarde no sistema e renove só quando o catálogo envelhecer.

3 **Faça poucas requisições.**
5 s por token (todas as rotas somadas) é o mínimo técnico, não a frequência de uso. Operação a cada 30 s ou mais, uma chamada de cada vez.

SDTS · Diretoria de Logística e Finanças · Outubro de 2026

1 Acesso e autenticação

Toda leitura usa o mesmo endereço e o mesmo token. Só `GET /` dispensa token.

HEADER PRINCIPAL
```
Authorization: Bearer cbm_live_...
```

ALTERNATIVA EQUIVALENTE
```
X-API-Key: cbm_live_...
```

> **Token na query string não autentica.** O token não tem prazo de validade; só deixa de funcionar se for revogado. Se perder o e-mail, peça outro — ele não pode ser recuperado.

Formato das respostas

As leituras vêm do banco da API, não de uma consulta ao CAD a cada GET. Todas seguem o mesmo envelope:

CAMPO	SIGNIFICADO
`dataset` / `categoria`	Qual conjunto foi lido
`fonte`	Sempre `"banco"`
`atualizado_em`	Hora da última carga daquele conjunto, em UTC. **Se for igual ao que você já tem, nada mudou.**
`total`	Quantidade de itens em `dados`
`dados`	O conteúdo em si

2 Rotas

API principal — a que o sistema acompanha OPERAÇÃO

ROTA	RETORNA
`GET /ativos`	Chamadas e recursos ativos juntos. Se um bloco falhar, o outro ainda vem; o bloco com falha traz `erro`.
`GET /ativos/chamadas`	Só chamadas ativas (categoria `ativas`). `/ativos/chamadas/ativas` é o mesmo conjunto.
`GET /ativos/recursos`	Viaturas e recursos em operação.

O dado operacional é o objeto `dados`. Telefones de solicitante e de envolvidos não vêm na resposta. Quando o `idRecurso` existe, cada empenho da chamada pode vir enriquecido com: `prefixo`, `dataHoraAtivacao`, `idUnidadeServico`, `placaMeioMaterial`, `complementoPrefixo`, `prefixoMeioMaterial`, `codigoSituacaoRecurso`, `componentes` e, se houver, `idMotivoIndisponibilidadeRecurso`.

Tabelas auxiliares — só para traduzir CARREGAR UMA VEZ

ROTA	USE PARA RESOLVER
`GET /dados-auxiliares/orgaos`	O órgão citado na chamada ou no recurso
`GET /dados-auxiliares/naturezas`	A natureza citada na chamada
`GET /dados-auxiliares/unidades-servico`	`idUnidadeServico` e a unidade do recurso
`GET /codigos`	As siglas da API principal (dicionário fixo, abaixo)

Rotas de apoio IMPLANTAÇÃO

ROTA	USO
`GET /status`	Diz se cada conjunto existe e quando foi atualizado. Consulte na implantação. **Não use como polling.**
`GET /` (sem token)	Só saúde do serviço. Não retorna dados.

3 Dicionário de siglas — `GET /codigos`

CHAVE → CAMPO	VALORES
`origem_acionamento` codigoOrigemChamada	**L** ligação/telefone · **I** integração · **B** bombeiro/interno · **A** automático
`SituacaoChamada` codigoSituacaoChamada	**E** em andamento · **A** aguardando/atendimento em aberto
`empenho_situacao` codigoSituacaoEmpenhoRecurso	**TR** terminado · **ED** em direção · **EN** encerramento · **DP** despachado · **AC** a caminho · **ER** em retorno · **NL** no local · **LI** no local intermediário · **EC** em controle · **SU** suspenso
`chamada_atendimento_situacao` codigoSituacaoAtendimentoChamada	**ER** em retorno · **NL** no local · **CL** classificada · **DP** despachada · **EC** em controle · **EN** encerramento · **LI** local intermediário · **ED** em direção · **TR** terminada · **AO** atribuída ao órgão

4 Códigos de resposta

Nenhum destes se resolve repetindo a chamada rapidamente.

CÓDIGO	SIGNIFICA	O QUE FAZER
401	"Credencial inválida." — token ausente, errado ou revogado	**Pare.** Peça outro token por e-mail. Não insista com o mesmo.
404	O conjunto ainda não foi gravado. Não é falha do token.	Avise o responsável pela API. Não repita a chamada.
429	O token chamou cedo demais (header `Retry-After`)	Espere o tempo de `Retry-After` e só então repita. Não reduza o intervalo.
503	Controle de uso indisponível	Tente mais tarde, com intervalo longo, sem laço rápido.

5 Passo a passo de integração

Relembrando as três regras. Token do e-mail fica em cofre ou variável de ambiente — nunca na URL, em log, em print ou em repositório. Auxiliares só traduzem códigos e ids da API principal: carregue uma vez. 5 s por token é o mínimo técnico, não a frequência de uso.

5 s	≥ 30 s	poucas / dia
Limite técnico por token. Nunca como frequência.	Leitura da operação (`/ativos`)	Renovação dos catálogos auxiliares

IMPLANTAÇÃO · UMA VEZ

1. **Guarde o token**
   Em cofre ou variável de ambiente (ex.: `CBM_API_TOKEN`), fora do código-fonte.
2. **Prove a conexão**
   `GET /codigos` com o header. 401 = token não enviado ou diferente do e-mail.
3. **Carregue os auxiliares**
   `/codigos`, `/orgaos`, `/naturezas`, `/unidades-servico` — uma de cada vez, mais de 5 s entre elas. Grave as quatro localmente.
4. **Consulte `/status` uma vez**
   Sem chamadas ou recursos? Avise o responsável e não entre em laço.

OPERAÇÃO · CICLO CONTÍNUO

5. **Leia a API principal**
   `GET /ativos` (ou só `/chamadas` ou `/recursos`). Junte ids aos catálogos locais e siglas a `/codigos`.
6. **Agende a próxima leitura para ≥ 30 s**
   Compare `atualizado_em` com o salvo; se igual, não há dado novo.
7. **Renove catálogos só quando antigos**
   No máximo algumas vezes ao dia, nunca a cada ciclo.
8. **Trate erros sem acelerar**
   429: durma o `Retry-After`. 401: pare e peça outro token.

6 Exemplos

TESTAR A CONEXÃO · CURL

```
export CBM_API_TOKEN='cole-aqui-o-token-do-email'   # ou carregue do seu cofre

curl -s \
  -H "Authorization: Bearer $CBM_API_TOKEN" \
  https://apicadbm.bombeiros.mg.gov.br/codigos
```

200 com o dicionário de siglas = conexão ok. 401 = token não enviado ou diferente do e-mail.

INTEGRAÇÃO MÍNIMA · PYTHON + REQUESTS

Carrega os auxiliares uma vez, consulta `/status` uma vez e lê `/ativos` a cada 60 s. Toda chamada passa por `chamar()`, que garante o espaçamento e trata 401, 404, 429 e 503.

```
import os, time, requests

BASE_URL = "https://apicadbm.bombeiros.mg.gov.br"
TOKEN = os.environ["CBM_API_TOKEN"]   # nunca escreva o token no código

INTERVALO_MINIMO   = 6          # entre QUAISQUER chamadas (limite técnico: 5)
INTERVALO_OPERACAO = 60         # /ativos: 30 s ou mais
RENOVAR_CATALOGOS  = 6 * 3600   # catálogos: poucas vezes ao dia

AUXILIARES = ["/codigos", "/dados-auxiliares/orgaos",
              "/dados-auxiliares/naturezas", "/dados-auxiliares/unidades-servico"]

class TokenInvalido(Exception): pass
class ConjuntoAusente(Exception): pass

_sessao = requests.Session()
_sessao.headers["Authorization"] = f"Bearer {TOKEN}"
_ultima = 0.0

def chamar(rota):
    """Uma chamada por vez, sempre espaçada. Não registre headers em log."""
    global _ultima
    for _ in range(3):
        espera = INTERVALO_MINIMO - (time.monotonic() - _ultima)
        if espera > 0:
            time.sleep(espera)
        _ultima = time.monotonic()
        r = _sessao.get(BASE_URL + rota, timeout=30)

        if r.status_code == 200:
            return r.json()
        if r.status_code == 401:
            raise TokenInvalido("Credencial inválida: pare e peça outro token por e-mail.")
        if r.status_code == 404:
            raise ConjuntoAusente(f"{rota} ainda não foi gravado: avise o responsável.")
        if r.status_code == 429:
            try:
                retry = float(r.headers.get("Retry-After", 30))
            except ValueError:
                retry = 30
            time.sleep(max(retry, INTERVALO_MINIMO))
            continue
        if r.status_code == 503:
            time.sleep(120)   # controle de uso indisponível
            continue
```

```
        r.raise_for_status()
    raise RuntimeError(f"{rota}: sem resposta válida após tentativas espaçadas.")

catalogos = {}

def carregar_catalogos():
    for rota in AUXILIARES:
        catalogos[rota] = chamar(rota)   # chamar() já espaça as requisições
    # grave os catálogos no banco/arquivo local do seu sistema aqui
    return time.time()

def versao(resp):
    """Identifica a carga pelo atualizado_em (no topo ou em cada bloco de /ativos)."""
    if "atualizado_em" in resp:
        return resp["atualizado_em"]
    return tuple(sorted((k, v.get("atualizado_em"))
                        for k, v in resp.items() if isinstance(v, dict)))

def processar(ativos):
    ...   # junte os ids aos catálogos locais e as siglas a catalogos["/codigos"]

def main():
    carregado_em = carregar_catalogos()
    status = chamar("/status")   # sem chamadas/recursos? avise e encerre

    ultima_versao = None
    while True:
        if time.time() - carregado_em > RENOVAR_CATALOGOS:
            carregado_em = carregar_catalogos()
        ativos = chamar("/ativos")
        v = versao(ativos)
        if v != ultima_versao:   # só reprocessa se houve carga nova
            ultima_versao = v
            processar(ativos)
        time.sleep(INTERVALO_OPERACAO)

if __name__ == "__main__":
    try:
        main()
    except (TokenInvalido, ConjuntoAusente) as e:
        print(e)   # não tente de novo nem entre em laço
```

Usando só `/ativos/chamadas` ou `/ativos/recursos`, o `atualizado_em` vem no topo da resposta e `versao()` já cobre o caso.

7 O que não fazer

- ✕ Colocar o token na URL — query string não autentica e expõe o token.
- ✕ Fazer várias requisições ao mesmo tempo ou rotas em paralelo com o mesmo token.
- ✕ Atualizar órgãos, naturezas, unidades ou `/codigos` a cada ciclo da operação.
- ✕ Tratar 5 segundos como frequência normal de atualização.
- ✕ Ignorar um 401 e continuar repetindo o token antigo.
- ✕ Guardar o token no frontend público. A chamada à API sai do seu backend.